

MettaKernel: A MeTTa-hosted Proof/Inference Kernel for both Dependent Type Theory and Higher-Order Tarski–Grothendieck Set Theory

Design, Curriculum, and a Prototype-First Build Plan

Oruži

Zar Goertzel

2026-06-18 (design draft); realized-architecture update 2026-06-23;
Metamath-profile update 2026-08-06; NIK/native-theory update 2026-08-14

Abstract

We set out to build one small *trusted* proof/inference kernel, written *in MeTTa* (as an ordinary MeTTa program, executed on the CeTTa engine), that can express and check proofs in *both* foundations we care about: CIC-grade *dependent type theory* (the foundation of Lean and Coq) and *higher-order Tarski–Grothendieck set theory* (HOTG, the classical higher-order logic of Megalodon). The implemented $\lambda\Pi$ checker is one important realization, but the theory has since become more general: the Native Inference Kernel (NIK) is a logic-polymorphic family of authorities and realizations, while the MeTTa Inference Kernel (MIK) is its MeTTa-hosted realization. A separately derived MeTTa-native type theory is one guest and one possible bootstrap language, not the definition of NIK. The kernel is a MeTTa program — not a new CeTTa language substrate — in the same spirit as `pverify`, an existing Metamath proof checker written in MeTTa. Three principles govern the work. First, **prototype a fully working kernel before any deeper integration into the engine**, so that the design is chosen empirically rather than frozen in advance. Second, **conversion is the danger line**: the kernel’s $\beta\eta\delta\iota$ conversion must be its own small, decidable, resource-bounded MeTTa functions and must never be delegated to CeTTa’s general runtime rewriting, or proof checking ceases to be a terminating judgment. Third, **use the native calculus licensed by each guest’s proved structure**: compute judgments directly when decidable, compose admitted sound operations without interior rechecking, preserve native proof objects when they are genuine mathematical data, and request certificates only at explicit search or trust boundaries. We present the two reference kernels we mirror (the Lean `PureKernel` for DTT and Megalodon’s `tp/tm/pf+check_doc` core for HOTG), a graded *curriculum* of the easiest examples that pin down the smallest fully expressive kernel (a DTT ladder following Smolka & Brown’s *Introduction to Computational Logic*, and an HOTG ladder following Megalodon), and a prototype-first build plan whose first milestone is an honest *capacity ledger* (AGREE/DIVERGE/OPEN) rather than a claim of finality. The MeTTa kernel’s verdicts are treated as a *subject* to be measured against the reference authorities, not as an oracle itself. We also state the proved relationship among evolving Zero/Prime/HE/PeTTa dialect points, intrinsic `PureKernel`, the locally nameless Pattern Pure presentation, proof-relevant NIK, and the strictly descending bootstrap tower. The account distinguishes closed formal interfaces from executable fragments and from remaining adequacy obligations.

1 Why a single kernel for two foundations

Lean/Coq rest on the Calculus of Inductive Constructions: dependent function and pair types, an inductive-definition mechanism with computation rules, a universe hierarchy, and a *decidable* definitional equality. Megalodon rests on HOTG: simple types with an impredicative `Prop`, proof terms,

classical logic, set membership, extensionality, foundational ($\in$ -)induction, and a choice operator. These are different mathematics, but a *proof kernel* for either is small: it is a term/type/proof object language plus a conversion relation plus a checking judgment, with everything else (parsing, automation, optimization, the ambient runtime) living *outside* the trusted core.

Our thesis — developed at length in the companion memo *metta_proof_kernel_grounded* — is that MeTTa is an apt host for such a kernel *as a program*: terms, types, and proofs are MeTTa atoms; the checking and conversion judgments are MeTTa functions; theories and imports are MeTTa data. The trusted waist is then a small auditable MeTTa module, and the two foundations appear as two *checkers* sharing that waist. We are not proposing to encode both foundations into one ambient rewriting theory (the Dedukti route); we keep the foundations’ judgments first-class and only share the machinery beneath them.

What this document is. A design + curriculum memo to drive a prototype-first build. It records: the reference kernels we mirror (§2); the MeTTa-program architecture (§3); the graded curriculum that defines “smallest fully expressive” (§4); the build plan and its first milestone, a capacity ledger (§5); the conversion discipline that is the central hazard (§8); the open questions we deliberately leave open (§9); and — added in this revision — the *realized architecture and results* (§6) recording the as-built kernel that now exists, including where realization refined the plan; and the generalized proof-aware architecture (§7), which records what is now formalized about NIK, the MeTTa-native type theory, dialect regions, and their exact boundaries.

2 The two reference kernels (calibration authorities and templates)

2.1 DTT: the Lean PureKernel

A trusted, *sorry-free* dependently-typed kernel already exists in Lean (`Mettapedia/.../MeTTa/PureKernel/`, 28 files). Its term language is

```
PureTm n ::= var (Fin n) | const DeclName | U0 | U1
          | pi A B | sigma A B | id A t u
          | lam B | app f a | pair t u | fst p | snd p | refl t
```

with two universes ($U0 : U1$), Π , Σ , and identity types. It provides one-step $\beta/\delta/\iota$ reduction, Church–Rosser confluence, `HasType`, a *decidable* definitional equality (`defEqByNormalization?`), subject reduction, and a bidirectional checker. Ordinary families (`Bool`, `Nat`, `Unit`) are *not* hard-coded: they enter through a general declaration mechanism (`DeclSpec`, `InductiveDecl`, `RecursorDecl`) with full well-formedness proofs. This is both our DTT calibration authority and our design template; crucially, the existing runtime-to-kernel bridge marks `directExecEquivalence` as `false`, so the runtime is *not* yet known to agree with this kernel.

2.2 HOTG: the Megalodon mathdata core

Megalodon’s trusted core (`repos/megalodon-1.13/src/mathdata.mli`) is genuinely compact — three small inductive types and a checker:

```
tp ::= TpVar i | Prop | Base i | Ar(tp, tp) | TpAll tp          (* simple types + type-vars
+ type-forall *)
tm ::= DB i | TmH h | Prim i | Ap(tm, tm) | Lam(tp, tm)
      | Imp(tm, tm) | All(tp, tm) | TTPAp(tm, tp) | TTPLam tm | TTPAll tm
```

```

pf ::= Gpa h | Hyp i | Known h | PTmAp(pf,tm) | PPfAp(pf,pf)
      | PLam(tm,pf) | TLam(tp,pf) | PTPAp(pf,tp) | PTPLam pf

```

plus `check_doc` (the proof checker), $\beta\eta\delta$ normalization with resource limits (`BetaLimit`, `NonNormalTerm`), and hash-addressed `doc/theory/signa` imports. Logic and set-theoretic content are *not* in this core: `True/False/not/and/or` are impredicative definitions, and membership/extensionality/choice/ $\in$ -induction enter as *axioms* in a preamble (Egal mode uses `Eps` choice and Tarski’s axiom; Mizar mode uses `the` and ZF-style axioms). The seed file `Mettapedia/megalodon/experiments/01_basics.mg` proves eight propositional theorems entirely by proof-term lambdas (e.g. `andEL := fun A B H => H A (fun a b => a)`).

2.3 Precedent that MeTTa can host a checker

`hyperon/metamath/pverify` (with `lib/pverify-utils.metta`) is a Metamath proof checker realized as a MeTTa program. It establishes feasibility of the *host*; it is Metamath, not HOTG, so it is precedent, not evidence about the present design.

3 Architecture: the kernel as a MeTTa program

Shared waist (written once in MeTTa). Atom encodings for binders (de Bruijn or locally nameless), α -safe substitution, typed contexts, declarations (parameter / definition / axiom / theorem / inductive / import), proof objects, the **conversion engine** ($\beta/\eta/\delta/\iota$ as explicit, terminating, resource-capped MeTTa functions), a **check/infer** driver, and hashing/provenance for imports. The two checkers below are *instances* of this waist — that sharing *is* the early unification.

HOTG checker. Mirror `tp/tm/pf` + `check_doc` + $\beta\eta\delta$ as MeTTa atoms and functions. Proof-term checking over impredicative `Prop`; classical/set/TG axioms are a *profile* layered above the core, admitted only after the bare proof-term checker is green.

DTT checker. Mirror `PureTm` (`U0/U1`, `II`, `$\Sigma$` , `Id`) with bidirectional `infer/check`, $\beta/\delta/\iota$, and decidable definitional equality by normalization; inductive families via `DeclSpec`, never hard-coded. Where the CeTTa engine’s `-profile he-prime` already provides dependent type inference, the MeTTa kernel may use it; new C features are added to `he-prime` *only* when the prototype proves a concrete need.

4 Curriculum: the examples that pin the smallest fully expressive kernel

The curriculum *defines* “fully expressive”: the smallest kernel is the one that checks every example below. The DTT ladder follows the chapter order of Smolka & Brown’s *Introduction to Computational Logic* (a Coq course by Megalodon’s own author), which introduces kernel features in increasing dependency order. The HOTG ladder follows Megalodon.

L0 — shared waist (both ladders ride on it). Names/binding, α -safe substitution, typed contexts, declarations, proof objects, conversion ($\beta/\eta/\delta/\iota$, decidable), hash imports + provenance, negative tests.

Table 1: DTT ladder (ICL spine + CeTTa **he-prime** seeds + PureKernel).

Step	Example (source)	Kernel feature it pins
D1	bool/negb/andb/andb_com (ICL §1.1)	inductive Bool, match/recursor, def-eq by computation
D2	plus_0/plus_S/plus_assoc (ICL §1.3–1.4)	Nat, recursor, structural induction
D3	Π / proofs-as-programs (ICL §1.2, 2.2, 2.6)	Π formation/intro/elim, application
D4	Σ / pairs (ICL §6.6; he-prime TutPackHuman)	Σ formation/intro/projection
D5	Id/refl/cong/transport (ICL §3.3; he-prime TutEq/TutRefl/TutSuccCong)	identity types, J /transport
D6	app_assoc/length_app (ICL §8)	parametric inductive + recursor
D7	Vec/TutIndex safety; VecN -1 (open)	indexed inductive families; index-domain question (Number vs Nat)
D8	Even/ $\leq$ (ICL §7)	inductive predicates, elim restriction
D9	recursors + elim restriction + universes (ICL §5)	ι -reduction, proof irrelevance, universe consistency

Table 2: HOTG ladder (01_basics.mg + Megalodon + mathdata tp/tm/pf).

Step	Example (source)	Kernel feature it pins
H1	propositional encodings (01_basics.mg, 8 thms)	tp(Prop), tm(All, Imp, Lam, Ap), pf(PLam, PPfAp, Hyp), β -normalization, check_doc
H2	$\forall$ -instantiation; $\rightarrow$ /proof application	PTmAp, PPfAp
H3	proof λ ; $\beta\eta\delta$; definition unfolding	δ on ThyDef/SignaDef, η
H4	equality/congruence (Leibniz eq)	polymorphic eq via TpAll
H5	membership/set/TG axioms (In/Empty/EmptyE, set-ext, In_ind, Eps)	<i>axiom profile</i> , admitted only after H1–H4 green

Cross-profile smoke. $P \rightarrow P$; $P \rightarrow Q \rightarrow P$; $P \wedge Q \rightarrow P$; $\perp \rightarrow P$; equality reflexivity/congruence; pair projections; and *one* tiny CeTTa operational-semantics theorem treated as a *certificate*, never as definitional equality.

Negative corpus (a fail-by-default gate — the soundness backstop). Bad de Bruijn index; unknown Known/Hyp; application type mismatch; a proposition-shape used as a proof object; cyclic/non-normal δ ; **an arbitrary MeTTa rewrite admitted as conversion** (the central danger); stale import/hash; VecN ...-1 wrongly accepted. The kernel must *reject* every item here; a checker that accepts any of them is unsound.

5 Prototype-first build plan

Phase 0. Shared MeTTa waist + the ledger harness (three deciders: MeTTa-kernel vs. Lean PureKernel reference vs. `he-prime C`).

Phase 1 (HOTG prototype). MeTTa `tp/tm/pf` + proof-term checker + β ; *check the eight 01_basics.mg theorems green* (H1–H2).

Phase 2 (DTT prototype). MeTTa `PureTm` + bidirectional checker + $\beta/\delta/\iota$ + decidable def-eq; *D1–D5 green*; ledger vs. the Lean reference authority and `he-prime C`.

Phase 3 (unify + harden). Both checkers share the waist; cross-profile smoke green; *negative corpus rejected*.

Phase 4 (climb the ladders). HOTG H3–H5 (definitions, equality, then set/TG axioms); DTT D6–D9 (lists, `Vec`, inductive predicates, recursors/universes); resolve the `VecN` index-domain question.

Later. Only after a fully working prototype and its lessons: consider deeper substrate integration — add the `he-prime C` features the prototype proved necessary; refine CeTTa. Not before.

Milestone 1 (the ledger, not a constitution). The deliverable is the working H1 + D1–D3 MeTTa kernel plus a populated *capacity ledger* with buckets AGREE-ACCEPT, AGREE-REJECT, DIVERGE, OPEN, plus the negative-corpus rejections. Divergences and open cases are first-class outputs, not blemishes to hide. Only then does external review become useful, and its question is whether the fragment boundary is honest and which divergences are bugs versus legitimate language semantics — not whether the final kernel is settled. The MeTTa-kernel verdicts are labelled **subject**; Lean PureKernel and Megalodon are the **calibration authorities**.

6 Realized architecture and results (June 2026)

The prototype is built and runs. This section records the *as-built* kernel and, honestly, where realization *refined* the plan above; the earlier sections stand as the original intent.

One $\lambda\Pi$ -modulo checker; object logics as signatures. Rather than two separate term languages (PureTm and `tp/tm/pf`), the realized kernel is a *single* $\lambda\Pi$ -style logical-framework checker (`Srt/Con/Var/Pi/Lam/App/Def`) with one `infer/check/conv`; each object logic enters as a *signature* of typed declarations over that one checker. DTT, HOTG, and HOL all appear in `kernel/curriculum_lf_demo.metta` as signatures over the same checker, each with positive proofs and a rejected negative. This keeps every logic’s rules first-class (as signature declarations) while sharing the trusted waist — LF-hosting, deliberately distinct from desugaring every logic into one ambient rewrite theory.

One generic inductive engine (DIndG). The hard-coded inductive families of the DTT ladder (D6–D9) are subsumed by a single generic *indexed-inductive* declaration, **DIndG**: a telescope IR (parameters Δ , indices Ξ , constructors) from which the family kind, constructor types, the eliminator, branch types, and constructor-headed ι -rules are *generated and checked*. `Nat`, `Bool`, `List`, `Tree`, `Pair`, `Vec`, `Fin`, and `Id` are all instances; crucially `Id` is itself a **DIndG** and `J` is its *generated* (diagonal) recursor, so the equality primitives and six special-case inductive slices retired onto the

one engine. (This resolves the original `VecN`/index-domain open question: indices are an arbitrary telescope, not a single `Nat`.)

Binding as a declared property (the ABT waist). Binding lives in one import-free waist (`kernel_binding_waist_v1`): de Bruijn under the hood, with capture-avoiding `shift/subst` written once and *arity-parameterized*; clients delegate to it rather than re-implementing substitution. A `BindingDecl` layer (`kernel_binding_decl_v1`) lets each hosted `LanguageDef` *declare* its binders (`Bind0/Bind1/BindFields/BindPi`: dependent Π , $\forall/\exists$ bodies, comprehension predicates, proof binders) without disturbing the hot waist. Capture-avoidance is mutation-gated.

Admission, checked-once theorems, and the admit-once lesson. A signature is *admitted* only if every declaration is well-formed: constants’ types check, definitions’ bodies check against their declared types (`wf-decl`), and inductive declarations are fully generatable. Two declaration forms serve derived libraries: `DAxiom` (an honest postulate, recorded in *axiom provenance*) and `DThm` (a *checked* theorem constant — its proof body is checked *once* at admission, then used opaquely by its type, transparent only for provenance). A memory hazard forced the central engineering lesson: naively, `kernel-check` re-ran full signature admission on *every* theorem check, so a file checking 21 theorems against a signature carrying two large derived-proof bodies re-normalized those bodies 21 times and exhausted memory (~ 48 GiB, OOM). The fix is *admit-once batch checking* — admission computed once, lexically, for a batch of jobs, with *no* forgeable “checked-signature” token exposed (in `MeTTa` a token is plain data anyone could fabricate). This is exactly HOL-Light’s discipline (a derived rule is a program calling primitives, never one giant proof term re-normalized from scratch) and it dropped the same file to ~ 8 GiB.

Faithful object-logic hosting: HOL-Light’s equality kernel. HOL is hosted not as a convenient natural-deduction surrogate but on HOL-Light’s *actual* primitive basis — `REFL`, `TRANS`, `MK_COMB`, `ABS`, `BETA`, `ASSUME`, `EQ_MP`, `DEDUCT_ANTISYM` as LF rule certificates checked by the imported kernel — with $\top/\wedge/\Rightarrow$ defined over equality and λ (matching `bool.ml`) and derived `MP/DISCH` shards admitted as checked `DThms`. Each primitive and each shard carries a mutation tooth. *Faithfulness caveat*: the definitional equations are currently *postulated* as constants matching `bool.ml` rather than introduced through a checked conservative-definition principle, and the derived `MP/DISCH` are specific checked replays, not yet a fully general derived-rule library.

The curriculum as a calibration matrix. The curriculum is now an explicit ledger in the `Curriculum` directory: each row is a kernel capacity $\times$ profile (`DTT/HOL/HOTG/Metamath/...`) $\times$ the SOTA system that calibrates the verdict (`Lean/Coq/Agda/Megalodon/HOL-Light`), with status `CHECKED` or `PLANNED` and a per-row gate. The external systems are the *calibration authority*; the destination is to *host each core logic as a LanguageDef and re-prove the curriculum’s theorems in our own kernel*. Verdicts are differential — a row borrows its expectation from the source system, not from our own guess.

The Metamath profile (August 2026 update). The Metamath row of the matrix is now the furthest along the stated destination. The full statement language is authored twice at the `LanguageDef` level — once in `languageDef!` form (`metamathCore`, including the front-end lowering pipeline as rewrite rules) and once as the proof-facing `sourceGrammar`, whose presentation passes the generic validator and whose accepted pipeline runs derive the outer database sort in the lexicalized language of their own classified streams with no supplied lexical

premises (`runSource_selfLexicalized_derives`). On the semantic side, every accepted source declaration transition is matched by the corresponding shipped `mm-lean4` database operation, and the whole statement fold co-evolves with the implementation database from the initial states (`default_initial_applyPayloads`), including the identification of the implementation's mandatory-frame trim with the specification's mandatory frame (`trimFrame'_eq_mandatory`). Checked source admission runs exact bytes through the actual verified reader into one generated presentation package. Deliberately open, in the obligations ledger's sense: per- $\$p$ proof discharge from the reader's own chronology (normal lane recently constructed; compressed in progress), the distinction between verified and ?-incomplete outcomes as disjoint results, and the prefix-wise source/reader simulation in which reader microsteps are administrative. The Metamath lane thereby serves as the template for how the other profile rows are intended to mature.

Validation discipline. Every checkpoint runs on three engines — CeTTa (C), PeTTa (Prolog), and the Lean-verified *LeaTTa* as a deterministic reference — in parity, with per-feature mutation gates (corrupt the rule, require red) and the fail-by-default negative corpus, all behind one `run_all` gate; heavy runs are memory-capped.

Soundness status (honest). Admission plus provenance close the per-input declaration holes (a `break_probes` probe documents what the bare term-checker would accept *without* admission; the real kernel rejects those). The *one* genuinely-open soundness obligation is metatheoretic, not a per-input check: **universal subject-reduction, confluence, and termination of the generated ι -rules are not yet machine-proven** — we have an executable SR-wedge self-test and the LeaTTa engine-floor, not the theorem. That metatheorem is the live Lean target. The kernel is today *checked-correct per input* but not yet *proven-correct as a calculus*.

7 From one hosted checker to a proof-aware NIK and native MeTTa theory

The runnable logical-framework kernel above motivated a more general theory, but it is not the definition of that theory. Three concerns remain distinguishable for derivation and audit:

NIK	laws of authorities, evidence, admission, and transport
MIK	MeTTa realizations of selected authority laws
guests	native MeTTa DTT, Pure, Megalodon/HOTG, Metamath, HOL, ...

This list is a decomposition of obligations, not a vertical system architecture or a sequence of foreign services. An authority fixes a claim family, its intended meaning, and how acceptance is established. A realization computes or checks that acceptance. The guest fixes the object logic. Consequently NIK has a metajudgment about accepted evidence, but no privileged object logic. The native MeTTa type theory is both a guest and the intended language in which language definitions, translations, proof operations, and lower-level authorities are represented as ordinary typed values. Megalodon and `PureKernel` remain independent calibration authorities rather than components silently baked into the NIK waist.

The target system boundary is therefore one reflective Prime, physically realized by CeTTa:

+-----+			
Prime			
LangCode objects	Morphism L L' values	Data_L(A)	
translations	proof objects	receipts	

spaces and Need	typed admission operations	
	NIK = the typed admission membrane	
+-----+		
	CeTTa = physical realization	

At the present formal boundary the language objects, structural language arrows, and their covariant Data action have semantic representatives as dependent terms in the selected Prime CwF. Surface definability of the whole GSLT-IL operation fragment and internal admission operations remain open. The diagram is the integration criterion, not a claim that those remaining theorems are already proved.

7.1 Dialects are points in a specification region

The names Zero and Prime no longer denote immutable semantic records. The Lean development separates a region of admissible operational points from the current branded point. Zero's region consists of occurrence-indexed operational points with bag observation attached separately. Prime selects an operational base, a bag observation, and a revision authority; the present query-first Zero backend is one assembly. Need, reflection, proof hosting, and the authored presentation are independently attached layers. Typed translations relate the current Zero and Prime points, and negative theorems show where those arrows cannot be strengthened: reflection contributes steps absent from the base, Need can escape the base image, and the Need route cannot in general be erased back to occurrence evaluation.

The native-type-theory derivation uses two inputs which must not be conflated. An *observation bag* records properties of a current dialect point: collection results, metavariable patterns, native reflection, first-class spaces, inert unknowns, costs, evidence, or staging. A separate *target-commitment* channel declares capacities the future family is required to gain: dependent-family reasoning, identity elimination, typed language manipulation, native proof-object programming, stratified bootstrap, and a direct certificate-free typing path. The theorem `commitments_extend_observations` proves that the commitments are not being laundered as observations of today's Prime. The direction of fit is therefore explicit: Prime may change to meet the commitments; the theory is not weakened to describe only the current implementation.

The current bags remain auditable authored data. Connecting each entry to a live dialect capability witness and an exact runtime parity theorem is a remaining realization task. Until then, the bags guide derivation and reject brand-fixing, but are not an automatic extraction of the CeTTa implementation.

7.2 The derived MeTTa-native type theory

The formal spine is a contextual, multimodal, intensional dependent type theory. It contains a category-with-families style context/substitution discipline, Π - and Σ -types, identity with J , dependent and binary inductive families, noncollapsed Tarski-style universes, typed quotation through explicit locked contexts, level-indexed staging, cost grading, proof-relevant evidence, and first-class codes for language presentations. Types-as-spaces is represented by an interpretation into predicates on Patterns with an explicit subtyping order; cost and evidence remain finer decorations and are not erased into mere truth sets.

Equality follows a proved profile architecture:

- The initial core has intensional, computational conversion.

- Behavioral equality is an explicit modal proposition or named profile, not silent kernel conversion.
- Function extensionality is not judgmental in the core.
- K/UIP is declared per profile.
- Cubical structure and univalence remain possible future authorities and do not burden every core judgment.

A concrete observational profile is proved to be a strict extension of the core; old derivations transport into it, the reverse inclusion fails, universes remain distinct, and an anti-leak theorem prevents tag erasure from turning profile equality into core conversion. The small Boolean profile used in this proof is a calibration witness, not the selected production equality algorithm.

The named audit object now has a concrete nondegenerate witness for every required field and its `openObligations` list is empty. This closes the abstract derivation audit, not the entire implementation project. The executable `FastPatternTyping` authority is a direct, certificate-free fragment over the live `PeTTa MeTTaType`; the full proof-relevant native typing doctrine has assumption, weakening, substitution/cut, evidence multiplicity, and proof erasure. What remains is one production-strength Pattern elaborator and conversion engine whose executable judgment has exact parity with that full declarative doctrine.

The status rule is deliberately memorable:

`zero obligations = abstract audit, not production checker`

Emptying the derivation ledger means that every field of the abstract candidate has a nondegenerate mathematical witness. It does not by itself supply a production elaborator, decide full Pattern conversion, or prove an implementation adequate to the declarative judgment.

7.3 Language-indexed Data: quotation, hosting, and dialect change

Quotation is not a synonym for the top type and not a hidden calling convention. The derived target separates three notions which older MeTTa dialects partially conflate:

`PDynamic = ?Unknown` (gradual unknown), `Atom = ⊤` (inert value top),
`DataL(A)` (held code of language *L* and type *A*).

Gradual consistency is nontransitive (for example, `Number ~ ?Unknown ~ String` without `Number ~ String`), whereas subtyping through the inert top is transitive. Evaluation control is a separate modal axis. Spaces may select an evaluate-by-default or data-by-default ambient polarity; explicit quotation holds in either space, and the surface `!` is the counit which evaluates already admitted Data in either space.

The surface keeps object variables and elaboration metavariables visibly distinct: `$x` is an object variable, `?a` is a solvable meta-level variable, and `?Unknown` is the distinguished never-solved gradual type. The inherited spelling `%Undefined%` denotes that same gradual type; it does not introduce a second type or a compatibility wrapper. Provenance such as “declared unknown”, “widened after conflicting uses”, or “introduced at a boundary” belongs to evidence or blame data, not to the kernel type lattice.

The formal type-code closure is deliberately non-idempotent: `Data (Data A)` is not `Data A`. Its explicit level strictly rises at each quotation, so code which writes code and the stratified bootstrap

tower retain their stage distinction. In each selected language fibre, the concrete modality is the product comonad $S_L \times -$, where S_L is a stage/space/revision stamp, not an execution trace. The counit projects an already-admitted payload and comultiplication retains the stamp while requoting. A Boolean-stamp canary proves that comultiplication is not surjective, so this is not an identity modality in disguise.

A typed language translation maps base type codes, their inhabitants, and stamps, then acts through every nested **Data** layer. Identity and composition are proved for both type codes and inhabitants, and the associated covariant Grothendieck construction has canonical strongly cocartesian push-forward lifts. Consequently the currently proved structure is an opfibration-style family with chosen cocartesian transport; the stronger packaged claim “split opfibration” is withheld until its cleavage coherence is stated at that abstraction layer. The word “fibration” is used only informally for the family: no contravariant pullback, and hence no genuine fibration theorem, is claimed without additional structure. This abstract family is instantiated on the actual category of validated language definitions and structural GSLT morphisms: a fibre’s base codes are exact authored sorts and its inhabitants are actual closed, well-sorted object terms at those sorts. Structural morphisms map both the sort and the held syntax while preserving its typing derivation; the resulting dependent family has functorial Data transport and the canonical strongly cocartesian lift for every authored route. Thus the index is not a two-language enumeration, an unstructured string, or a unit-valued witness which erases the program. The live two-point instance additionally uses the existing Zero-to-Prime operational translation on semantic-state type indices, preserves the indexed state exactly, has a strongly cocartesian promotion lift, and has no reverse Prime-to-Zero push-forward.

NIK governs entry into these fibres; it does not turn the counit into replay. Direct decisions, native proof kernels, and admitted flows all have no separate mandatory certificate language. Only an explicitly selected certificate boundary does. Once a value has entered **Data**, **!** does not inspect which admission mode established it. This is the formal separation between a trust membrane and ordinary typed execution.

The four inherited HE meta-types **Symbol**, **Variable**, **Expression**, and **Grounded** are represented as disjoint refinements of held syntax rather than as evaluation-changing supertypes. Their classifier is total and single-valued, all four refinements have concrete inhabitants, and a grounded hook is proved not to inhabit the expression refinement. This keeps the useful syntax taxonomy while removing its control-flow side effect.

The first authored dialect delta is also formal. In value position HE **Atom** maps to Prime’s inert top; in an argument domain its historical hold-before-evaluation convention maps to **Data** **_**. Call-site elaboration is preserved exactly: the translated signature inserts the hold which HE obtained from the parameter type. The partial inverse is proved on precisely the image of the translation, and a negative theorem shows that Prime’s gradual dynamic type has no fabricated HE preimage. Constructor translation changes constructor heads only; de Bruijn indices and free metavariable names are fixed pointwise, free-name sets are preserved, and well-scoped source syntax remains well scoped. More strongly, translation commutes with the canonical locally nameless opening, closing, lifting, and binder-eliminating substitution operations. Open code is now packaged as actual authored **OpenTerms** indexed by an exact free-variable typing context, an ordered de Bruijn type stack, and an authored carrier sort. Language/context pairs and context-preserving structural routes form a category; their fibres contain these open terms, and their Data action is functorial on both type codes and inhabitants. A Zero-to-Prime witness transports a genuinely open Atom variable while retaining its name and mapped type. An out-of-range bound index is rejected before quotation, and a target context which does not classify a free name cannot receive that name through transport. Thus the contextual box transports without weakening or capture; a

nonidentity constructor translation remains the negative control showing that hygiene was not proved by making translation trivial. These are the mathematical gates for a future **cetta -trans he.prime** prime realization, not a claim that the command-line realization already exists.

Surface hygiene contract. An open quotation is formed only from a judgment $\Gamma \vdash_L t : A$ and records L , Γ , and A in its contextual Data type; raw text or an untyped Pattern is not a quotation premise. Free object names are exactly those classified by Γ , while bound variables use locally nameless indices and canonical binder metadata. A language route $T : L \rightarrow L'$ transports the whole judgment to $T\Gamma \vdash_{L'} Tt : TA$: it changes authored constructors and types, but fixes free names and bound positions and commutes with opening, closing, lifting, and capture-avoiding instantiation. Splicing requires an explicit context map into the surrounding context; accidental name coincidence supplies no authority. The counit **!** applies only to already admitted contextual Data, and does not convert failed admission or incomplete guest checking into evaluation. Ambient space polarity may insert quotation or evaluation at elaboration boundaries, but it cannot alter these typing and capture conditions. The final surface spelling may infer recoverable indices, but it must elaborate to this judgment and may not weaken it.

This construction is a specification ablation as well as a proposal. Collapsing **?Unknown**, **Atom**, and **Data** destroys the proved nontransitive gradual-consistency witness, makes a nominal upcast change evaluation behavior, and removes the separate held-syntax index. Making **Data** idempotent destroys the strict-level theorem. Removing language indices loses typed guest hosting and language-changing transport. Requiring a certificate at the counit contradicts the certificate-free direct and admitted-flow modes. The surviving design is therefore selected by independent positive and negative laws rather than by the spelling of today’s dialect.

7.4 How GSLT/MeTTa-IL and Prime properties determine the surface

The theories retain different jobs, but those jobs must not become different external products. A GSLT language definition supplies authored syntax, equations, and one-step dynamics. MeTTa-IL supplies an indexed language in which definitions, translations, and execution routes are themselves manipulable data. These objects and operations must be internal to Prime: users must be able to store, match, translate, mine, and chain over them with the ordinary Prime language. Prime also supplies the operational commitments which must survive that abstraction: open-world inertness, collection-valued answers, first-class spaces, revisioned Need, costs, evidence, and reflection. The native dependent theory supplies the contextual typing discipline which says when those manipulations are safe. Finally, NIK is the admission membrane between an arbitrary proposed value and an accepted inhabitant inside the same system. The derivation flow may be displayed externally for analysis, but the intended operational organization is:

```
Prime {
  languages      : LangCode values
  translations    : Morphism L L' values
  held syntax    : Data_L(A)
  logic          : typed GSLT/MeTTa-IL operations
  dynamics       : spaces, match, Need, cost, and evidence
  boundary       : NIK admission of proposed operations and values
}
Cetta realizes these operations; it does not host a separate GSLT service.
```

This internalization has four theorem-level gates. First, an internal transport combinator must have a Prime type and its denotation must be exactly the already proved functorial action on Data.

This semantic gate is now constructed in the selected Prime CwF: source and target languages, their structural route, and held source syntax are dependent term arguments; application computes the Data push-forward; identity and composition hold; and the same route action commutes with opening, closing, lifting, and instantiation. A concrete closed Prime term supplies a positive inhabitant, while the impossible identity-symbol map from Prime back to Zero is the negative control. Exposing this term through the final surface grammar and elaborator remains part of the definability gate, not a reason to externalize it. The gate now also has a contextual instance: language/context pairs, their context-preserving routes, and open held syntax are dependent inputs to a second Prime semantic term. Its result is indexed by the target context, preserves the exact free-name support, and proves that an absent target name cannot appear. This supplies the open-code half of I1 without pretending that semantic inhabitation already proves surface expressibility. Second, the GSLT-IL operation category must correspond fully and faithfully to a definable Prime fragment, so neither direction is merely an encoding convention. Third, admission and its optional receipts must be Prime operations and values, with the C path proved to realize them rather than bypass them. Fourth, Prime’s own language definition must be a transportable, inspectable `Data` value at a strictly higher level. A standing design test follows: a proposed meta-operation is not part of GSLT-IL until it has a Prime-internal representative of the stated type. The existing hygiene and functor laws now inhabit that representative; the reverse direction of the definability theorem and the other gates remain open.

This architecture changes ergonomics because effects formerly hidden in a broad type become visible, typed term formers. The target operator boundary is:

Surface notion	Native target meaning	Implementation status and consequence
<code>Atom</code>	inert top of the ordinary value order	The current authored Prime language still uses broad <code>Atom</code> fields. Removing evaluation control from this type is a migration target, not a completed source change.
<code>@/quotation</code>	introduction of language-indexed held syntax $\text{Data}_L(A)$	Current Prime already has structural quotation. The type index, contextual hygiene, and cross-language action are proved in the separate Data layer, not yet one serialized production operator.
<code>!</code>	counit/evaluation of an already admitted Data value	The Lean operation is proved check-free after admission. The present langdef spells the crossing as <code>prime-evaluate-name</code> ; adopting <code>!</code> as uniform surface sugar still requires elaborator and runtime work.
<code>Need</code>	demand scheduling under a revisioned authority	It remains a runtime control operator. It does not become proof checking and does not require an interior certificate trail.

space polarity	evaluate-by-default or data-by-default elaboration mode	This controls where quotation/evaluation syntax is inserted; it does not identify <code>Data</code> , gradual unknown, or top, and it does not alter the kernel judgment.
----------------	--	---

Thus the core mathematical operators do change at the *native target*: holding and running become a typed pair, and language translation becomes an operation on held syntax. The live Prime langdef has not yet been destructively rewritten to those spellings. Compatibility profiles can preserve HE/PeTTa calling conventions while a typed translator inserts the explicit operations. The important ergonomic invariant is that a user pays syntax at a boundary, not checks on every interior reduction.

Lean4Less supplies an independent external calibration of the equality-profile architecture. It translates selected ordinary Lean terms to a smaller Lean-minus profile by replacing definitional proof irrelevance and K-like reduction with explicit transports, then checks the result with a specialized Lean4Lean kernel. This follows the extensional-to-intensional translation line of Winterhalter et al. The bounded NIK integration skeleton accordingly treats ordinary Lean and Lean-minus as two native proof-system guests with exact proof-fibre parity, connected by a judgment-preserving profile translation and a noncollapsing universe-order embedding. It does not claim that the external translator has discharged these laws for all Lean: the audited implementation still reports scaling limits, possible transport explosion, deliberately aborted declarations, and a currently broken `.olean` output route. Lean4Less is therefore evidence that explicit profile transport is a real architecture, not a completed Lean-as-NIK instance.

7.5 Pure is an exact source fragment, not the owner of the native theory

There are two distinct Pure developments. The intrinsic `PureKernel.HasType` uses de Bruijn terms, declarations, universes, $\Pi/\Sigma/\text{Id}$, conversion, and inductive infrastructure. The Pattern presentation uses locally nameless terms and the extrinsic judgment `PureHasType`. The native-theory development now proves a substantial relationship for the intrinsic side:

```
intrinsic PureKernel
-- faithful, exact on its image; substitution and typing commute -->
native raw terms and the native contextual support
```

The embedding has projection, round-trip, image, and injectivity theorems. Renaming and substitution commute with it, typing is exact on its image, and the typed context/substitution map is faithful. It is not full: staged substitution and quotation provide named native terms with no typed preimage in intrinsic Pure. Thus the native theory conservatively retains the Pure fragment while strictly extending it with Pattern collections, staging, costs, evidence, and language codes.

The other side of the triangle is now sharply delimited, but not yet closed. The two authored typing judgments are not merely notational variants. `PureKernel.HasType` omits domain or codomain well-formedness premises from several introduction and elimination rules which `PureHasType` records explicitly. For example, the raw intrinsic relation types a lambda whose domain is the untyped top universe; a new regular intrinsic spine rejects that derivation and forgets soundly into the raw relation. Conversion in the regular spine is intensional, but conversion to an ordinary target now requires a derivation that the target is a type; conversion to the untyped top sort is a distinct rule. Raw telescopes are likewise separated from regular contexts: the raw syntax

admits a top-sort assumption and variable typing consumes it, while the regular context formation judgment rejects the same telescope.

This selection is no longer merely editorial. The regular rules are packaged as a rule-model interface, and the inductive regular judgment maps into every relation closed under that interface. It is therefore the *least* rule-closed relation generated by the stated finite rules. The inclusion into raw `PureKernel.HasType` is proved, and three independent ablations prove that the reverse inclusion fails: raw typing accepts a lambda with an unformed codomain, reflexivity over an unformed carrier, and a dependent pair whose declared domain is the untyped top universe. The regular rules reject all three. Removing any corresponding premise therefore strictly enlarges the calculus.

The structural metatheory now proves why these local repairs compose. Regular conversion commutes with renaming and declaration-free simultaneous substitution; regular typing is stable under renaming, weakening, substitution, and singleton instantiation; and formation of the components of syntactic Π , Σ , and identity codes is invertible. Consequently, in every regular context, each typing result is either the distinguished untyped sort `U1` or is itself typed by `U1`. Presupposition closure is a global theorem of the judgment, not a review convention attached to selected rules.

Two tempting categorical slogans are deliberately withheld. An inductive typing calculus is a least fixed point of finite derivations, not a greatest fixed point; the latter would admit cyclic or infinite evidence. And “coreflection” is earned only after a category of presentations and the required adjunction are actually defined. The present theorem is the precise initial/least-closure property in the preorder of typing relations, not an unproved adjunction.

The prospective native kernel should therefore enforce context and type regularity by construction, with `PureKernel.HasType` retained as a permissive raw presentation rather than silently crowned as the final `MeTTa` judgment.

The syntax boundary has also been made exact. Declaration-free intrinsic terms quote into `PureTmPattern`, and a tagged constant quotation has `Pattern-purity` if and only if its source is declaration-free. A negative constant witness is proved. The exercise additionally exposed a real defect in the legacy artifact quote: it renders a declaration name directly as an application label, so a declaration printed as `U0` collides with the reserved universe constructor. The tagged quote removes that collision while agreeing byte-for-byte with the legacy quote on the constant-free fragment. The exact boundary no longer uses the declaration-name pretty-printer at all: names are encoded structurally by their anonymous, string-component, and numeric-component constructors. This encoding is proved injective. The whole exact term quote is consequently proved injective under an injective, binder-compatible variable environment; the binder case reopens the two locally closed bodies and uses the close/open cancellation law. Regular context formation implies that every telescope entry is in the common syntax, and equality of quoted regular contexts now reflects to equality of intrinsic telescopes. Thus syntax and contexts cannot hide a mismatch. What remains is the full intrinsic–extrinsic typing equivalence for the regular conversion-bearing spine.

The forward half is now proved. A substitution-derived change-of-opening-name theorem transports both conversion and typing across the cofinite binder names, without postulating alpha equivalence. Every declaration-free intrinsic reduction constructor maps to `Pattern` conversion; an explicitly fragment-internal conversion proof maps recursively; and every regular intrinsic typing derivation, including dependent formation and elimination, maps to `PureHasType`. The identity function supplies a positive end-to-end witness. The conversion proof fibre needed strengthening during this proof: requiring only constant-free endpoints was insufficient, because raw symmetric conversion can leave the fragment through a term containing a declaration constant and later return to a pure endpoint. The replacement relation certifies every reduction edge, reflexive endpoint, and intermediate term as declaration-free. A concrete constant-bearing beta detour proves that this restriction has content.

Unrestricted reflection into the regular intrinsic spine is now proved false, even when all displayed terms lie in the exact constant-free image. The authored Pattern conversion rule can assign $U0$ a beta-redex type convertible to $U1$ without requiring that redex itself to be a well-formed type; the regular intrinsic judgment rejects exactly that derivation. Therefore the reverse square must use a Pattern boundary whose presuppositions are intrinsic, or an elaboration into the regular intrinsic kernel. It may not be obtained by silently identifying the two authored judgments. Reflection for that corrected boundary remains open. “Pure” should consequently name an admissible region of related intrinsic and presented authorities, not whichever encoding was implemented first.

The sound computational conversion component of that corrected boundary is now available. One-step and finite reduction are proved to preserve the declaration-free fragment; complete development therefore yields a fragment-internal conversion proof. The executable regular comparison uses the same complete-development computation as the earlier checker, but returns the strengthened conversion relation whose every edge, endpoint, and intermediate term remains in the common fragment. Positive universe reflexivity and negative cross-universe computations prevent the comparison from being vacuous.

The old one-pass comparator is not a complete decision procedure. A declaration-free two-step redex is proved fragment-internally convertible to a variable while the current one-pass development comparator returns failure; this lifts the older raw incompleteness observation into the regular proof fibre. A production regular checker must therefore use a genuinely complete normalizer before it can supply a **DecidedRelation**. The executable part of that replacement now exists: an outermost-leftmost selector carries a proof of every chosen reduction, failure is equivalent to irreducibility, and accessibility of the reduction relation drives normalization to a unique normal form. On the explicit domain “reduction-accessible and declaration-free”, this induces an exact certificate-free **DecidedRelation**; a beta-redex reducing to $U0$ is the positive computation and $U0/U1$ is the negative computation. The self-reducing declaration-free omega term is proved outside the domain, so syntax purity cannot masquerade as termination. The required logical-relations theorem is now proved: every subject admitted by a full regular judgment is reduction-accessible. This promotes the existing normalizer from an explicitly supplied accessibility domain to every admitted regular subject. Building a production bidirectional checker which constructs the full regular judgment remains a separate realization task. The first reducibility layer for that proof is now formal. Accessibility is closed under dependent function, dependent pair, identity, lambda, pair, and reflexivity constructors, and it reflects from applications and projections to their components. Scoped reducibility candidates carry CR1–CR3, admit intersection, nondependent function-space, and product-space constructions, and have concrete positive and negative inhabitants. In particular, the identity lambda inhabits the normalizing function candidate while the delta self-application does not. A separate theorem proves why this layer cannot be replaced by structural induction: two individually accessible delta components form the non-accessible omega application. Reduction steps now also reflect through arbitrary variable renamings, so accessibility forms a renaming-stable family of candidates across de Bruijn scopes; injectivity of the renaming is not assumed. This is exactly a presheaf-level result, not yet a Kripke model. A scope-aware function candidate now quantifies over every future renaming and every argument admitted in that future scope. Its candidate laws are proved using a fresh-variable probe, reflection of reduction through renaming, and accessibility induction on arguments. The construction includes positive identity and negative looping-delta witnesses.

The first argument-indexed extension is also formal. A dependent codomain selects a reduction candidate from the actual argument and supplies the exact backward conversion transport needed when that argument reduces. Constant codomains recover the nondependent function candidate definitionally. A second saturated candidate—strongly normalizing terms which never reduce to

$U0$ —then supplies a genuinely nonconstant codomain: $U0$ belongs to the fibre indexed by $U0$ but not to the fibre indexed by $U1$. Thus argument dependency is witnessed rather than represented by a constant-family placeholder. At this intermediate layer, substitution naturality and the interpretation of syntax were not yet available; the subsequent semantic-universe construction supplies them.

The need for that stronger layer is forced by a separate negative theorem: the open self-application is accessible, and the substitution sending its variable to the accessible delta term is pointwise normalizing, yet the instantiated term is omega and is not accessible. The proof object must therefore be a genuinely dependent, type-indexed candidate semantics for contexts and candidate-respecting substitution; this identifies the fundamental-lemma boundary rather than an unfilled evaluator case. The constructor-specific head-expansion interface needed at that boundary is now formal as well. Exact beta expansion requires accessibility of the lambda body, the argument, and the instantiated contractum; first- and second-projection expansion require accessibility of both pair components. These laws are stable under semantic weakening and reindexing, and are required only over environments realizing their contexts. They cannot be replaced by generic backward closure: an erasing beta redex with the looping omega term as its argument reduces to $U0$, although the redex itself is not accessible. Thus the fundamental lemma must consume the exact introduction laws, not assume that reducibility is closed under arbitrary reverse reduction. A realized-context refinement now states the candidate laws only for substitutions which realize their semantic context. Each such environment still yields an ordinary reduction candidate with the exact head-expansion interface, while context extension no longer demands semantic candidate data for malformed tails. The earlier coherent candidates embed conservatively: their normalizing predicates agree pointwise. Contextual extension, weakening, reindexing, head-variable realization, and positive/negative normalization canaries are proved. This is the interface needed for a genuinely dependent function candidate whose closure proof may consume context realization. That candidate is now constructed: its codomain is selected in the context extended by the actual argument, and argument reduction uses both directions of environment coherence. CR1–CR3, renaming, base-environment reduction, and exact beta and projection expansion are proved. The identity lambda is a positive inhabitant; the accessible delta function is rejected because its self-application is omega. The later semantic-universe layer interprets the syntactic type formers with this candidate, and the simultaneous fundamental lemma below consumes these exact laws. The dependent pair candidate is now constructed at the same realized-context boundary. A pair is observed through both projections: the first projection is checked in the base environment, while the second is checked in the environment extended by that actual first projection. Pair reduction therefore moves both the observed second component and its codomain environment. CR1–CR3, renaming, both directions of environment-reduction coherence, and exact beta and pair-projection expansion are proved. Pair introduction explicitly transports the second component from the environment indexed by the written first component to the environment indexed by the pair’s first projection. A pair of $U0$ terms is the positive witness; a pair with omega as its second component is rejected.

Universe, dependent-function, dependent-pair, and identity codes now have a proof-relevant semantic-universe relation retaining their exact contextual candidate meanings. Semantic contexts retain the candidate selected for every telescope entry. Candidate-respecting substitutions lift through dependent extensions, support comprehension and singleton instantiation, and commute extensionally with both dependent-function and dependent-pair construction. Semantic type codes pull back along every such substitution: the transported meaning is extensionally the reindexing of the source meaning. This is the substitution theorem required by language-internal transport, not an external checker correspondence. Conversion is also coherent with the retained proof-relevant meanings: convertible semantic type codes select extensionally equivalent contextual candidates.

The dependent-function and dependent-pair cases use Church–Rosser constructor injectivity and transport the dependent codomain across the semantic identity substitution induced by equivalent domain meanings. Consequently, a fixed semantic type code has a meaning unique up to the observable extensional equivalence, without erasing its dependency structure or identifying proof records definitionally.

The conversion guard needed by the simultaneous proof was strengthened during this construction. Raw fragment conversion does not transport normalization backward: an erasing beta redex containing omega converts to $U0$ while remaining non-normalizing. Merely requiring accessibility at the identity environment is also too weak for dependent substitution. Semantic conversion therefore requires both endpoints to normalize after *every* environment realizing the current semantic context. The guard is symmetric, transitive, and stable under semantic substitution. A beta-hidden dependent function code is the positive witness; looping and erasing-omega codes are negative controls. The target-formation premise of regular conversion is consequently load-bearing evidence for the logical relation, not redundant syntax. The simultaneous formation-and-membership fundamental lemma over this universe is now proved by induction on the regular typing derivation. It interprets formation and ordinary membership together, extends semantic contexts at dependent premises, and uses conversion coherence to transport terms without identifying proof records. During this proof, application and both dependent-pair projections exposed a further presupposition gap: their domain formation had previously been recovered from another premise. The regular constructors now require domain formation explicitly, so the proof is structurally recursive and the kernel judgment carries every premise its semantic interpretation consumes. Every regular context consequently has a semantic realization; every full regular judgment has an accessible subject. The identity program is the positive witness, while the self-applicative omega term is proved to have no regular judgment at any type. Identity witnesses require no stronger candidate in the present fragment, because identity has reflexivity but no eliminator. The normalization interpretation therefore uses the contextual normalizing candidate and proves its introduction rule from accessibility of the reflected payload. This is explicitly not a semantic identification of endpoints. Reflexivity of $U0$ is the positive witness; reflexivity carrying omega is rejected, so the interpretation does not hide divergence. The older algorithmic checker also returns the permissive authored typing relation, so it is not renamed or promoted: the new bidirectional checker must return the regular judgment and discharge formation premises on every fast path.

OSLF and LADL guide the next promotion step, but do not choose an authored typing judgment by themselves. OSLF supplies operational space and behavioral models in which each candidate premise can be tested; a rule is promoted only after its interpretation is sound and nondegenerate. Behavioral equivalence remains an explicit modal proposition or equality profile and never leaks into kernel conversion. LADL can derive collection-sensitive connectives only after the relevant term/collection distributive law is constructed. In the present theory, bags retain occurrence-relevant evidence while set quotients retain theoremhood; the distributive law connecting that distinction to the complete native syntax is still a named construction target, not an automatic consequence of the GSLT.

7.6 Four native modes of checking and reasoning

The NIK waist does not reduce every guest to transcript replay. It supports four modes, each with a different evidence boundary.

Mode	Native action	Evidence and cost boundary
------	---------------	----------------------------

Direct decision	A <code>DecisionKernel</code> computes a semantic judgment directly.	The induced checker has unit evidence; ordinary typing and decidable conversion need no certificate language.
Native proof kernel	The kernel computes the claim proved by the guest's own proof object.	Exact proof-fibre equivalence identifies accepted certificates with native proof objects, rather than merely decoding one direction.
Admitted operation flow	Operations proved once to preserve meaning and acceptedness compose accepted premises into accepted conclusions.	No observer or checker runs in the interior path; rechecking is needed only when an untrusted value crosses the boundary.
Certificate boundary	An untrusted searcher or importer supplies the finite choices which deterministic kernel computation cannot recover.	The kernel reconstructs consequences and checks once; receipts and exported proof artifacts are optional boundary products, not permanent runtime luggage.

direct decision	native proof	admitted flow	boundary evidence
$J(c)$ computed unit evidence	$p : c$ computed exact proof fibre	$f(p_1, \dots, p_n)$ sound once, flow freely	choices supplied reconstruct once
no replay	proof objects retained	no interior recheck	receipt/export optional
<i>One logic-polymorphic waist; the guest selects the mode. The modes may coexist within one authority at different trust boundaries.</i>			

Figure 1: The four NIK execution/proof modes. Certificate carrying is one boundary mode, not the ontology of NIK or the default typing path.

The exact-fibre requirement is essential. A checker may accept a proof object decorated with an ignored tag and still have a sound decoder, while carrying more accepted evidence than the native proof system. The formal ignored-tag counterexample rules out calling such a format canonical parity. Compressed or multi-encoding formats must instead provide an explicit normalization quotient.

The converse boundary is also formal. A computable step-budget checker gives sound and complete finite evidence for fixed-input halting, while no computable `DecisionKernel` decides the corresponding predicate. Certificates can therefore add genuine power at a search boundary, but they add nothing to a judgment already decided by a native kernel. Megalodon instantiates the middle of this spectrum concretely: its native proof kernel computes the result of a `Mathdata` proof object, has exact canonical proof-fibre parity, and decides the explicit fuel-bounded $\beta/\eta/\delta$ conversion relation. The fuel bound is part of the stated relation; this is not an unqualified normalization theorem.

Contexts and consequence are optional doctrines above this waist. Cartesian weakening and contraction are never forced on a resource-sensitive guest; named counterexamples prove that a

resource context can admit neither duplication nor discard. Proof erasure into a thin consequence relation and hence a π -institution-like view is likewise an optional reflection, not the internal logic of raw NIK and not evidence that GSLT-IL is the internal logic of $\text{Ind}(\text{ProofGSLT})$.

7.7 The Maximal-Native-Calculus Principle

The policy name is the **Maximal-Native-Calculus Principle**; its precise formal content is *structure-licensed maximal-element selection*:

For each guest and certified fragment, recognize a finite family of admitted calculus realizations, order it by a declared license order, and select a maximal licensed member. Do not flatten a guest to generic replay, and do not impose structural rules or completeness claims which its presentation does not earn.

This is now packaged as a theorem. A **RecognizedFamily** contains a finite recognized set, a nonempty licensed subset, and an admitted operation at every index. The declared order must be capability-monotone, and every strict comparison must exhibit a capability gained by the stronger realization, so an arbitrary index order cannot masquerade as calculus strength. Finiteness yields a maximal licensed index; selecting it adds no checker or replay layer, and semantic preservation follows from the selected admission arrow. “Maximal” does not mean “greatest”: a proved two-element discrete counterexample has two incomparable maximal choices and no greatest choice. Nor does it mean globally fastest: the license order concerns certified capability, while cost is a separate verified refinement.

The supporting theorem family places this selection between exact positive and negative bounds:

- direct semantic decision embeds as a full authority with no proof details;
- exact native-proof parity prevents certificate inflation;
- admitted proof operations compose and preserve acceptedness with no interior observer;
- resource guests refute an ambient cartesian default;
- the halting boundary refutes universal direct decision;
- general reachability analysis, dead-code elimination, nontrivial property-driven optimization, and universal optimal compilation meet explicit impossibility theorems; and
- every explicitly finite fragment has a correct optimal compilation witness.

These results yield a three-band engineering discipline:

A future calculus license should therefore be an admitted, revisioned claim about a guest or fragment: it names the required structural capabilities, the executable engine, its soundness and adequacy scope, and any completeness or cost result. It should not be one giant options record and should not enlarge the public NIK checking ABI. This is also the common algebra for proof-rule admission, verified MAM transformations, and Prime runtime specialization: a transformation proved once to preserve the relevant invariant may thereafter run on the fast path.

Decidable band	Compute the judgment directly.	No certificate language; use the admitted fast path after checking.
Searchable band	Retain only genuine choices made by search.	Reconstruct deterministic consequences and check once at the boundary.
Open semantic band	Stratify theories and authorities.	Do not promise a finite complete calculus, same-level self-certification, or a universal optimizer.
compute $\longrightarrow$ certify choices $\longrightarrow$ stratify as decidability decreases.		

Figure 2: The three-band discipline. Finite evidence reaches the recursively enumerable/searchable band, not arbitrary semantic truth.

7.8 Proof objects, GSLT-IL, and the bootstrap boundary

The native theory retains proof-relevant evidence rather than reducing it to truth. Proof objects may be inspected and transformed as native data; an admitted operation supplies the correct-by-construction path, while an arbitrary agent or external prover remains on the check-once boundary. The multisorted-clone structure records substitution and composition of such operations when the guest admits them. This gives a precise target for typed proof-object programming in Prime: typechecking a transformation should establish that it factors through admitted operations, after which its interior execution needs no repeat check.

Prime’s connection to GSLT-IL now has two rigorously separated layers. The live Need algebra is equivalent to the single returned-fibre fragment, with step equivalence, clone equivalence, and a commuting admission square. A negative witness shows that a pending claim is not encoded by that fragment. Above it, a new two-point indexed diagram makes today’s Zero and Prime distinct theory objects, maps the nonidentity Zero-to-Prime route to the existing operational translation, and proves that route application is simultaneously a GSLT-IL command step and a certificate-free NIK admission arrow. A concrete request crosses that language boundary, enters Prime Need, and then uses the admitted Need operation to flow correct-by-construction with no interior recheck. The reverse route is empty and the returned fragment still has no full command decoder. Thus the indexed bridge is nontrivial but cannot be mistaken for a derived implementation, scheduler, or arbitrary-theory language algebra.

The dialect-projection result is correspondingly split. HE, PeTTa, Zero, and Prime requirement bags are exact projections of one observed native requirement spine. This is not yet the desired authored-calculus equation

$$\text{DialectRules} = \text{filterMap}(\text{NativeRules}, \text{projection}) ++ \text{namedQuirkDelta}.$$

The Lean target interface requires a surviving shared native rule and a quirk delta strictly smaller than the dialect calculus. A negative theorem proves why an instance cannot be extracted from bags alone: HE and PeTTa have identical observed typing-property lists but their authored base typing calculi have 22 and 21 rules. The current PeTTa tree also contains distinct 50-rule determinism and 72-rule guard presentations. Regression totals such as 132/132 or 76/76 must not be relabeled as authored rule inventories. The exact three-dialect calculus projection awaits a common finite native rule presentation and explicit maps into each actual authored calculus.

Finally, the bootstrap metalogic enforces only what its index can honestly prove. A level- $n + 1$

authority may check a refinement or fidelity claim about a level- n kernel. Every edge strictly descends in level, and two-level certification cycles are impossible. This supports progressively smaller trusted bases and eventual verification of fast implementations without changing their runtime cost model. It does *not* prove same-level semantic self-consistency, and it supplies no provenance merely from the existence of a level number.

7.9 Space operators, quantified queries, and operational polarity

The fusion with Prime is not limited to placing a dependent checker beside an existing evaluator. Prime's spaces expose an algebra which the native theory must classify without turning search into definitional equality. At the raw collection level, a match has the schematic semantics

$$\text{match}(S, p, t) = \{ \{ t[\sigma] \mid a \in S, \sigma \text{ unifies } p \text{ with } a \} \}.$$

Thus filtering a pattern is bounded selection, applying the result template is image, shared variables across clauses induce joins, and multiplicity remains observable until an explicit set quotient erases it. Union, intersection, difference, update, aggregation, and finite least fixed points then form natural neighbours of match. This identifies a candidate safe query fragment and a large body of database and logic-program optimization laws, but completeness of an operator basis still requires an explicit syntax, semantics, and normal-form or expressivity theorem.

Universal matching is not merely a condition-bearing macro over ordinary match. Its primitive meaning is the space-inclusion judgment

$$\text{matchAll}(S, p) \iff S \subseteq \{ a \mid a \text{ unifies with } p \} :$$

every atom in the space must unify with the pattern. The finite executable contract is proof-relevant: on success it retains the complete multiset of bindings from every atom, including multiplicity; on failure it returns exactly the bag of nonmatching atoms. The success constructor remains distinct even for an empty space, so vacuous truth is not confused with evaluation failure. A result template may then be instantiated at every retained binding. Thus ordinary match asks whether some atom has the requested shape, while universal match establishes that the entire space has that shape. Selecting one violation gives cheap failure; demanding more enumerates the violation stream. A separate operator which applies an arbitrary Boolean condition to each match is derivable from match, branching, and aggregation. It must distinguish condition value **False** from failure to produce a Boolean value; the latter is **UNDETERMINED**, not a counterexample and never evidence of universal success.

Logically, this coverage query has the more precise shape

$$\forall a \in S. \exists \sigma. \text{Unify}(p, a, \sigma).$$

The flattened binding bag enumerates the inner existential witnesses with multiplicity; it is not by itself the dependent function witnessing the outer universal. Consequently **matchAll** is not automatically the categorical right adjoint of **match**.

This law does, however, define a genuine patterned refinement of spaces:

$$\Gamma \vdash S : \text{Space}[p] \iff \prod_{a \in S} \sum_{\sigma} \text{Unify}(p, a, \sigma).$$

The dependent product retains a separately chosen unification witness for each atom occurrence; it does not require one global substitution to fit the whole space. The executable finite checker is

adequate for this judgment, and the judgment is stable under reindexing of the ambient context. A surface form such as $(: \ \&S \ (\text{Space } p))$ is therefore coherent only when elaboration runs that checker and stores the resulting revision-keyed license. Merely parsing a type assertion is not enough. Conversely, a form such as $(\text{in } \&S \ p)$ normally reads as membership and would obscure the refinement judgment.

For an infinite intensional space, absence of a yielded violation proves nothing. When the space is the least fixed point of finitely presented generator rules, however, inclusion can be established by the pre-fixed-point principle: the pattern denotation must contain the base facts and be closed under every generator. Failure then returns a rule instance which breaks closure rather than requiring enumeration of an infinite atom. This is the induction principle for generated spaces, and it makes the native totality check a finite rule-level operation whenever the corresponding containment recognizer applies.

The Lean contract proves the finite characterization in both directions, the cache-polarity laws (removal preserves a universal fact; a bad addition returns that atom as its exact revocation witness), and the abstract least-fixed-point closure license. For logic programs the closure premise is decomposed into base facts plus preservation by every clause under every grounding. This is a semantic license, not an executable recognizer: a native fragment must still decide those quantified obligations and prove its acceptance sound, while recognizer failure yields INCOMPLETE, never refutation.

The truthful outcome depends on the admitted cardinality and generation mode:

- a materialized finite space may be scanned to exhaustion;
- a certified closure-finite rule space may first be saturated and then scanned;
- an open intensional space denotes a least fixed point and is explored on demand, but universal truth requires induction, model checking, or another admitted proof principle; and
- a bounded search returns a witness, a counterexample, or INCOMPLETE with its stated budget and coverage. Statistical or PLN evidence may decorate that last result, but it is not promoted to truth.

Cyclic derivations with an independently checked progress condition are one possible finite proof representation for inductive or μ -calculus claims; they belong at the proof-search boundary, not in ordinary evaluation.

This operator algebra is also the concrete customer for the future hyperdoctrine laws. For a context projection $\pi : \Gamma \times A \rightarrow \Gamma$, contravariant reindexing π^* has the desired adjoint chain $\exists_\pi \dashv \pi^* \dashv \forall_\pi$. Thus existential binding is the left adjoint and universal binding is the right adjoint of reindexing, not of one another and not of the raw match engine. In a proof-relevant fibre, $\forall_\pi$ returns a dependent product assigning every fibre element its witness; in the thin Boolean shadow it reduces to ordinary “all”. The finite `matchAll` contract is one decision procedure for the bounded $\forall\text{--}\exists$ predicate above. Beck–Chevalley and Frobenius laws would then justify moving quantified matches across substitutions and joins. Those laws must be proved for the actual bag-, evidence-, and resource-sensitive fibres before the native structure is called a hyperdoctrine.

Evaluation polarity is consequently a property of a space or elaboration region, not of the top type. Computation-oriented regions may evaluate by default; knowledge-oriented regions may hold by default. The stable surface separations are: $\$x$ is a MeTTa object variable; $?a$ is a solvable meta-level variable; $?Unknown$ is gradual imprecision; an inert top type makes no evaluation decision; language-indexed `Data` classifies held syntax; and $@/!$ are the quote/run introduction and elimination forms. A future universal-match surface and type-directed optional arguments are

elaborator choices, not additional kernel equalities. Position-free arguments are sound only when their expected types select a unique placement; otherwise names or tags are required.

7.10 Typed weighted temporal control

Three neighboring algebras must remain distinct. A graded clause changes the weight of an authored reduction and uses a commutative semiring, so alternative matches can be accumulated independently of enumeration order. A policy grades and reorders already-authorized occurrences using a quantale; its sequential product need not commute, because route order may matter. A modal μ -calculus objective describes temporal acceptance, and a parity progress certificate can justify a cyclic liveness argument. None of these objects authorizes a new reduction edge.

The generic controller preserves every live occurrence exactly. Therefore every finite run is sound, and any two controllers which both exhaust the same additive search have the same answer bag. Liveness is intentionally not policy-independent: a legal depth-first controller can starve an answer which a breadth-first controller reaches. Fairness, recurrence, and eventuality must be separate ν/μ objectives with checked progress evidence. Fixed-point bodies are admitted only after the existing positivity check; the concrete formula $\mu X. \neg X$ is rejected rather than passed to a parity authority.

Prime’s semantic CwF now contains policy, temporal-objective, and weighted parity-automaton values with their carriers visible in their types. The default Need policy elaborates to exactly the pre-existing breadth-first scheduler. PLN uses the already established order-dual parallel-evidence quantale directly: revision becomes quantale multiplication, retaining positive and negative support. When a graded **where** clause specifically requires a scalar, an explicit projection uses the already-proved finite-evidence equality

$$\text{strength}(e) \text{ confidence}_{\kappa}(e) = \frac{e^+}{e^+ + e^- + \kappa}.$$

Equal strength with unequal evidence volume yields unequal projected propensity, so confidence has not been erased. Cost, weakness, and other carriers remain separate quantale instances and may be related only by stated morphisms or product constructions.

The Boolean and quantitative accounts now share a proved boundary rather than parallel folklore. Proposition-valued quantale semantics is exactly the ordinary modal semantics, constructor by constructor, including fixed points. Consequently the existing polarity theorem for quantale-valued satisfaction specializes to Boolean positivity without a second structural induction. A positive fixed-point body denotes a Mathlib monotone order homomorphism on the powerset of states; its impredicative μ and ν clauses are respectively the Knaster–Tarski least and greatest fixed points. On a finite state carrier, both stabilize after at most the carrier cardinality: iterate from the empty set for μ and from the universal set for ν . These finite approximants are the semantic ranks required by the remaining parity-game adequacy proof.

Two boundaries remain open. An independently specified Prime surface grammar and elaborator must be proved to denote these semantic policy values; semantic inhabitation alone is not a full internal-language theorem. The positive modal μ -calculus compiler is now executable: it separates ambient observations from fixed-point back-edges, produces a finite verifier/falsifier arena, proves every generated pointer in bounds, proves that every binder back-edge reaches an actual introducing fixed-point node, and connects an executable finite model pointwise to the original denotational objective. Winning strategies include local move validity, so an illegal selected edge cannot win merely because it generates no play. A proof-side semantic table is structurally aligned with the node table, and finite least-entry and greatest-exit ranks connect fixed-point truth and

refutation to the approximants needed by the parity proof. The unit-weight embedding places any such arena in the existing quantale-weighted automaton type. As a nontrivial witness, the existing $\nu Y.(\mu X.O \vee \langle a \rangle X) \wedge [a]Y$ recurrence objective compiles to a nine-node cyclic arena; its denotation holds on a one-state observed loop, and an executable progress certificate proves its selected strategy parity-winning. The generic adequacy theorem

$$s \models \varphi \iff \text{the verifier wins the compiled game from } (s, \varphi)$$

remains to be proved. Thus concrete checked games are valid authorities, while arbitrary positive formulas are not yet promoted solely by compilation.

7.11 Categorical dimensions and the ultrainfinite boundary

Several categorical structures coexist, at deliberately different strengths. At dimension one, contexts and substitutions form a guest-chosen category; claims and accepted evidence reindex contravariantly. Language codes and structural translations form another category; the Data family varies covariantly and its Grothendieck total category has chosen strongly cocartesian push-forward lifts. These variances must not be conflated. The current Data theorem is opfibration-shaped: without contravariant pullback it is not a full fibration, and without packaged cleavage coherence it is not yet advertised as a split opfibration.

The neutral contextual evidence structure is the beginning of a proof-relevant indexed doctrine, not yet a full hyperdoctrine. A full hyperdoctrine would add logical structure in each fibre, adjoints to reindexing for quantifiers, and Beck–Chevalley/Frobenius coherence. Generic predicate-fibration and Beck–Chevalley infrastructure exists separately, while the native theory has a CwF-style comprehension and substitution spine. Instantiating the former on the actual native Pattern doctrine, with proof-relevant bags and resource-sensitive variants handled correctly, remains a theorem-level integration task. Proof erasure already has a precise categorical comparison: proof-relevant set-evidence reflects to thin closure through an adjunction, and indexed erasure yields an optional π -institution. Neither result makes thin consequence the definition of NIK.

At dimension two, an execution route retains its authored steps as data. A generating 2-cell compares two routes with the same endpoints; vertical composition and left/right whiskering freely build larger comparisons. Concrete naturality diamonds compare “reduce then transport” with “transport then reduce”, and Prime has a comparison between direct and lazy Need routes. This is useful proof-relevant two-dimensional syntax, but not yet a bicategory: unit and associativity coherence for 2-cells, interchange, and the appropriate quotient or higher witnesses remain open. In particular, the raw generated syntax is proved not to collapse a reflexive vertical composite definitionally, which prevents an unearned coherence claim.

“Ultrainfinite” names a two-variance ambient programme, not a completed $(\infty, 1)$ - or $(\infty, 2)$ -category. Filtered compact stages map into an ambient GSLT (the Ind-like growth direction), while a cofiltered atlas projects the ambient object out to compact perspectives (the Pro-like observation direction). Compact probes into a supplied filtered colimit factor through a finite stage; recovery of the ambient object from all shadows is a separate optional limit property. Ultrafilters then state perspective-relative verdicts, with principal collapse and reindexing laws, without turning a perspective into the whole object. The higher-categorical endgame would organize routes, transformations, transformations between transformations, and their coherences; the present formalization stops honestly at finite routes and generated 2-cells.

The bootstrap tower is orthogonal but compatible: its level index orients authority edges strictly downward, while the ambient charts organize growth and observation. One says which level may check which lower level; the other says how finite stages and perspectives relate to an ambient

system. Neither entails semantic consistency, reconstruction, or provenance. Their common design rule is that finite, carried data—proofs, routes, cells, stamps, and receipts—must remain available when operational use needs it, while its proof-erased or observational shadow is taken only by an explicit map.

The present bootstrap implementation is genuinely natural-number indexed. Its lowest contract uses a finite index below the host, its tower is assembled at successor levels, and the native quotation and universe witnesses use adjacent successors. A transfinite generalization is therefore not a global textual replacement of `Nat`. It should first extract a weak descent interface indexed by an arbitrary type with a well-founded strict relation:

$$\text{LowerContract}_{<}(h) = \sum_{\ell} (\ell < h) \times \text{Statement}(\ell).$$

Strict descent, absence of self-edges and two-cycles, and well-founded induction belong to this layer. Adjacent quotation and recursively generated universe levels require a separate successor or ordinal-recursion capability. The existing natural-number tower must then be recovered as a conservative instance before ordinal or Tarski–Grothendieck indices are installed. No density or arbitrary interpolation follows from well-foundedness, and no transfinite index supplies a semantic model or consistency proof by itself.

The descent-only interface has now been extracted and proves no self-edge, no two-cycle, composition of descent, and absence of infinite descent over any partial order with well-founded strict order. It also proves `StrictlyBelow n` equivalent to `Fin n`, rules out dense refinement, exhibits lexicographic administrative coordinates, and supplies an ordinal witness. This closes the index-independent descent core only; migrating the successor-indexed quotation and universe tower remains open.

7.12 Status boundary and division of exposition

At this revision the combined Lean modules for the NIK metalogic and native type-theory derivation build together, their public theorem audit uses only the project-permitted foundational axioms, and the named derivation obligation count is zero. In addition, finite maximal licensed-calculus selection, a concrete Zero-to-Prime indexed language-changing route, capability-level dialect projections, a language-indexed Data family with chosen strongly cocartesian push-forwards and four admission capabilities, the first exact HE-to-Prime Data delta, and a bounded Lean/Lean-minus NIK guest skeleton are now formalized. The regular-normalization line additionally has a presupposition-complete least typing relation, a global presupposition theorem, three independent negative premise ablations, and a structural renaming and substitution theory. It also has a scope-aware Kripke function candidate, an argument-indexed dependent extension, and a proved nonconstant codomain witness. Its semantic-environment layer now also has candidate-respecting substitutions, context comprehension, projection, pairing, and the two substitution beta laws. A stronger coherent layer proves that pointwise environment reduction acts through substitution, preserves context realization, and leaves the selected dependent candidate invariant. Exact beta and pair-projection head expansion is also proved at this coherent semantic layer, together with an erasing-omega counterexample to arbitrary reverse-reduction closure. The subsequent realized-context layer conservatively embeds those coherent candidates and proves contextual extension, weakening, and reindexing without assigning candidate semantics to malformed environments. Its dependent function candidate, including the exact constructor expansion laws and nondegeneracy canaries, is also proved. Above that layer, a proof-relevant semantic universe interprets the four current type-code formers, semantic contexts retain their candidate meanings, and semantic substitution pulls type meanings

back through dependent context extension. Its conversion boundary is uniformly normalizing in every realizing environment and is itself stable under semantic substitution. The universe is coherent under conversion: constructor heads are conversion-invariant, and convertible codes have extensionally equivalent meanings, including dependent codomains compared after semantic transport across equivalent bases. The simultaneous formation-and-membership fundamental lemma is proved on top of this coherence, yielding semantic realization of every regular context and strong normalization of every subject admitted by a full regular judgment; ω is the negative non-admission witness. The following claims remain outside that closure: Prime-internal morphism and admission terms satisfying the four internalization gates above; a production full-DTT Pattern checker; automatic extraction of dialect bags from live realizations; the exact authored-calculus projection-plus-delta theorem; the intrinsic-Pattern Pure typing reflection theorem for a regularized Pattern boundary beyond the now-proved syntax, proof-fibre, cofinite-renaming, and forward-typing results; the full arbitrary-theory Prime-GSLT-IL language algebra; discharge of the external Lean4Lean/Lean4Less guest package; automatic recognition of every calculus license; and verified staged transport down to the production C realization. The proposed complete space-operator basis, production universal-match realization, cardinality-mode recognizers, cyclic-proof authority, and actual quantifier adjunctions remain design targets. The finite proof-relevant universal-match contract, cache polarity, abstract closure license, and a one-sided closure-recognizer interface are now Lean theorems; they do not yet constitute the native operator or a complete recognizer family.

This paper is the home of the unified kernel/native-theory architecture. `ProofGSLT.tex` retains the narrower mathematics of proof presentations, open derivations, checked DAGs, substitution, transport, and wire formats. `CeTTa.tex` records the implementation consequences: direct typing, admit-once fast paths, boundary-only receipts, typed dispatch, and compiled authority realizations. Neither companion should redefine NIK as its own special case.

8 Conversion discipline (the central hazard)

The single biggest architectural risk is identifying kernel conversion with general MeTTa evaluation. We hold five lines: (A) decidable conversion $\neq$ general MeTTa eval — the kernel’s $\beta/\eta/\delta/\iota$ are its own bounded MeTTa functions (resource caps in the spirit of Megalodon’s `BetaLimit`); (B) MeTTa’s many-valued/nondeterministic typing vs. a functional DTT judgment — the kernel commits to a functional infer/check; (C) grounded/FFI values are a trusted boundary — grounded equality is not kernel reduction; (D) universe consistency is not optional forever, because proofs *about* MeTTa are a goal; (E) the kernel must be small and separable — a pure checking slice, not entangled in effectful evaluation.

9 Open questions (kept deliberately open)

Resolved or refined by the realized architecture (§6): the `VecN`/index-domain question (indices are an arbitrary telescope under `DIndG`); the first dependent-typing fragment ($\lambda\Pi + \text{DIndG}$ indexed inductives, predicative); the trust boundary (it is signature *admission*); and the convergence target (the kernel is its own $\lambda\Pi$ -LF, with `PureKernel`/Megalodon as templates and the Lean-verified `LeaTTa` as the runnable reference floor, rather than `PureKernel` itself).

Live and deliberately open: (1) the **universal subject-reduction / confluence / termination metatheorem** for the generated ι -rules — the single open soundness obligation of the realized LF calculus, now a Lean target (the toolchain is unblocked at v4.31); (2) **engine-level term sharing** (hash-consing / DAG / normalization-by-evaluation in `CeTTa`) for deeper scal-

ing, of which admit-once batch checking is the cheaper architectural half already in hand; (3) a **checked conservative-definition principle** so hosted definitional equations are derived rather than postulated; (4) a **fully general derived-rule library** (beyond specific checked replays); and (5) **breadth** — hosting further core logics as LanguageDefs (fuller HOTG via Megalodon, then Lean-core/CIC) over the now-shared, now-affordable waist.

For the generalized architecture of §7, the main open goals are complementary rather than replacements for that calculus theorem: (6) a full executable native Pattern DTT with exact parity to its declarative evidence doctrine; (7) live capability and parity witnesses for each advertised Zero/Prime/HE/PeTTa bag entry; (8) the intrinsic–Pattern Pure typing reflection theorem after making Pattern-side presuppositions intrinsic (raw `PureHasType` is now proved too permissive for equivalence, while the regular intrinsic side is globally presupposition-closed); (9) the indexed Prime–GSLT-IL language-manipulation algebra beyond the now-proved arbitrary validated-language Data family and contextual hygiene, especially authority-preserving action on language presentations themselves and executable dialect round trips; (10) recognizers and adequacy proofs that populate the finite maximal-calculus license layer for real guests; (11) the exact authored-dialect projection-plus-quirk-delta instances; (12) discharge of the Lean4Lean/Lean4Less guest package; and (13) a concrete staged refinement chain from the mathematical authority to the production realization; (14) the safe space-operator basis and its licensed relational rewrites; (15) native realization and differential qualification of the now-formal finite, intensional, and bounded universal-query contracts, including fragment-complete closure recognizers; and (16) migration of the current bootstrap tower onto the now-formal descent interface, while keeping successor-sensitive quotation and universe structure as an explicit stronger capability. None of these gaps licenses weakening exact proof-fibre parity, importing cartesian structure into resource guests, or claiming semantic self-consistency from the bootstrap index.

References (local anchors)

- G. Smolka and C. E. Brown, *Introduction to Computational Logic*, Lecture Notes, Saarland University, 2014. (DTT curriculum spine.)
- *metta_proof_kernel_grounded* (Aixiv, 2026): framework core, object-theory profiles, conversion discipline, hash imports.
- Megalodon: `repos/megalodon-1.13/src/mathdata.mli` (the HOTG `tp/tm/pf` + `check_doc` core); `examples/` (Egal/Mizar preambles); `experiments/01_basics.mg` (seed).
- Lean `PureKernel/README.md` and modules (the DTT reference/template).
- `PureKernel/RegularNormalizationBoundary.lean`, `RegularNormalizationAlgorithm.lean`, and `RegularReducibility.lean`, and `RegularKripkeCandidates.lean`, and `RegularCandidateSemantics.lean`, and `RegularCandidateCoherence.lean`, and `RegularHeadExpansion.lean`, and `RegularContextualCandidates.lean`, and `RegularContextualPi.lean`, and `RegularContextualSigma.lean`, and `RegularAccessibleConversion.lean`, and `RegularContextualIdentity.lean`, and `RegularStructural.lean`, and `RegularSemanticUniverse.lean`, and `RegularSemanticCoherence.lean`, and `RegularFundamental.lean` (exact conversion boundary, executable accessibility-driven normalizer, and the renaming-stable reducibility layer with substitution counterexamples, scope-aware function candidates, and the first genuinely argument-sensitive codomain,

followed by semantic context comprehension, candidate-respecting substitution, reduction coherence of dependent environment indices, and exact constructor head expansion with its negative arbitrary-reversal control, followed by the conservative realized-context candidate interface and its genuinely dependent function and pair candidates, together with the accessibility guard required for semantic conversion, the scoped normalization semantics for identity witnesses, the global presupposition and specification-ablation theorems, and the proof-relevant substitution-stable semantic universe together with extensional uniqueness of semantic meanings under conversion, the simultaneous fundamental lemma, and strong normalization of every fully regular judgment).

- `hyperon/metamath/pverify` (MeTTa-hosted checker precedent).
- J. Meseguer, “General Logics,” 1989 (proof calculi and covariant translation of proof objects; stronger conservativity is separate).
- B. Ahrens and P. L. Lumsdaine, “Displayed Categories,” 2019 (indexed object/morphism families without an assumed lifting property).
- T. Winterhalter et al., “Eliminating Reflection from Type Theory,” POPL 2019 (extensional-to-intensional translation by explicit transports), together with the `ett-to-itt` formalization and Lean4Less’s adaptation.
- `Mettapedia/GSLT/LanguageDef/KernelAuthority.lean` and `NIKMetalogic.lean` (direct kernels, exact proof fibres, contextual and resource doctrines, admitted operations, impossibility frontier, bootstrap).
- `Mettapedia/Languages/MeTTa/NativeTypeTheoryDerivation.lean` (dialect observations and commitments, native modal DTT, Pure refinement, equality profiles, Prime returned-fibre bridge, exact parity audit).
- `Mettapedia/Languages/MeTTa/Prime/DataFibration.lean` and `DataTranslationKernel.lean` (non-idempotent language-indexed Data, validated-language structural action, chosen strongly cocartesian push-forwards, four entry modes, surface-axis separation, exact first dialect delta, and contextual opening/closing/substitution hygiene).
- `ProofGSLT` names the class of proof-supporting GSLTs. The explicit generic checker is the *ProofGSLT reference realization*; these names are not interchangeable.